LAPORAN TUGAS SISTEM OPERASI (PRAKTIKUM)

TUGAS P5: SHELL SCRIPT

USER MANAGEMENT

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah : Sistem Operasi



Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA

JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA

POLITEKNIK NEGERI BANDUNG

2026

DAFTAR ISI

BAB I PENDAHULUAN **3**

1.1 Ringkasan Tugas 3

1.2 Lingkungan Pengerjaan 3

1.3 Rancangan Hak Akses 4

BAB II MEMBUAT GROUP DAN FOLDER GROUP **5**

2.1 Instruksi 1: Delapan Group dan Folder Group 5

BAB III MENGATUR PERMISSION **6**

3.1 Instruksi 2: Folder Group dan Folder Shared 6

BAB IV SHELL SCRIPT CREATE USER **8**

4.1 Membuat users.txt dengan vi 8

4.2 Menulis Script dengan vi 10

4.3 Isi Script dan Cara Kerjanya 12

4.4 Menjalankan Script 14

BAB V MEMERIKSA HASIL **18**

5.1 File users-created.txt 18

5.2 File actions.log 19

5.3 Group dan Folder Baru 20

5.4 User dan Home Folder 21

5.5 Uji Hak Akses 22

BAB VI PENUTUP **23**

6.1 Ringkasan Hasil 23

6.2 Lesson Learned 23

6.3 Kesimpulan 23

BAB I PENDAHULUAN

1.1 Ringkasan Tugas

Tugas P5 melanjutkan studi kasus P4 (user, group, dan permission), tetapi pembuatan user tidak lagi diketik satu per satu. Ada tiga instruksi: (1) membuat delapan group dan folder group di `/company`, (2) mengatur permission supaya folder group hanya bisa diakses group pemiliknya dan folder shared bisa dibaca semua group, dan (3) menulis shell script di vi yang membuat user dari file `users.txt`, memeriksa user dan group yang sudah ada, menampilkan pesan untuk setiap aksi, serta menghasilkan dua file log.

Setiap jawaban berisi command line, penjelasan, dan screenshot terminal asli. Seluruh langkah dijalankan berurutan di satu server, jadi kondisi tiap langkah melanjutkan langkah sebelumnya.

1.2 Lingkungan Pengerjaan

Supaya hasilnya tidak tercampur dengan user dan group dari tugas P4, dibuat server Ubuntu baru yang masih bersih di OrbStack bernama `ciwaruga-srv`. Semua perintah dijalankan oleh akun admin `ghaisan` yang punya hak sudo, di folder kerja `~/p5`, dengan zona waktu WIB.

Tabel 1.1: Versi perangkat lunak di server ciwaruga-srv

Komponen	**Versi**
Sistem operasi	Ubuntu 26.04.1 LTS (OrbStack)
Kernel	Linux 7.0.14-orbstack aarch64
Shell dan interpreter script	GNU bash 5.3.9
Text editor	vi (VIM 9.1)
`useradd`, `groupadd`, `usermod`	passwd (shadow-utils) 4.17.4
`setfacl`, `getfacl`	acl 2.3.2
`ls`, `chmod`, `chown`	uutils coreutils 0.10.0
`sudo`	sudo-rs 0.2.13

1.3 Rancangan Hak Akses

Rancangan permission sama untuk delapan group awal maupun group baru yang dibuat script, sehingga hasil akhirnya seragam:

Tabel 1.2: Rancangan pemilik dan permission

Path	Pemilik	Mode	Alasan
`/company`	root:root	755	semua user boleh melewati folder induk
`/company/<group>`	root:<group>	2770	hanya root dan anggota group (rwx); other tidak punya akses sama sekali; setgid membuat file baru ikut group folder
`/company/shared`	root:root	750 + ACL	tiap group mendapat entri ACL `r-x` (read-only), other ditolak; default ACL mewariskan aturan ke isi baru
`/home/<user>`	<user>:<user>	bawaan	dibuat otomatis oleh `useradd -m`

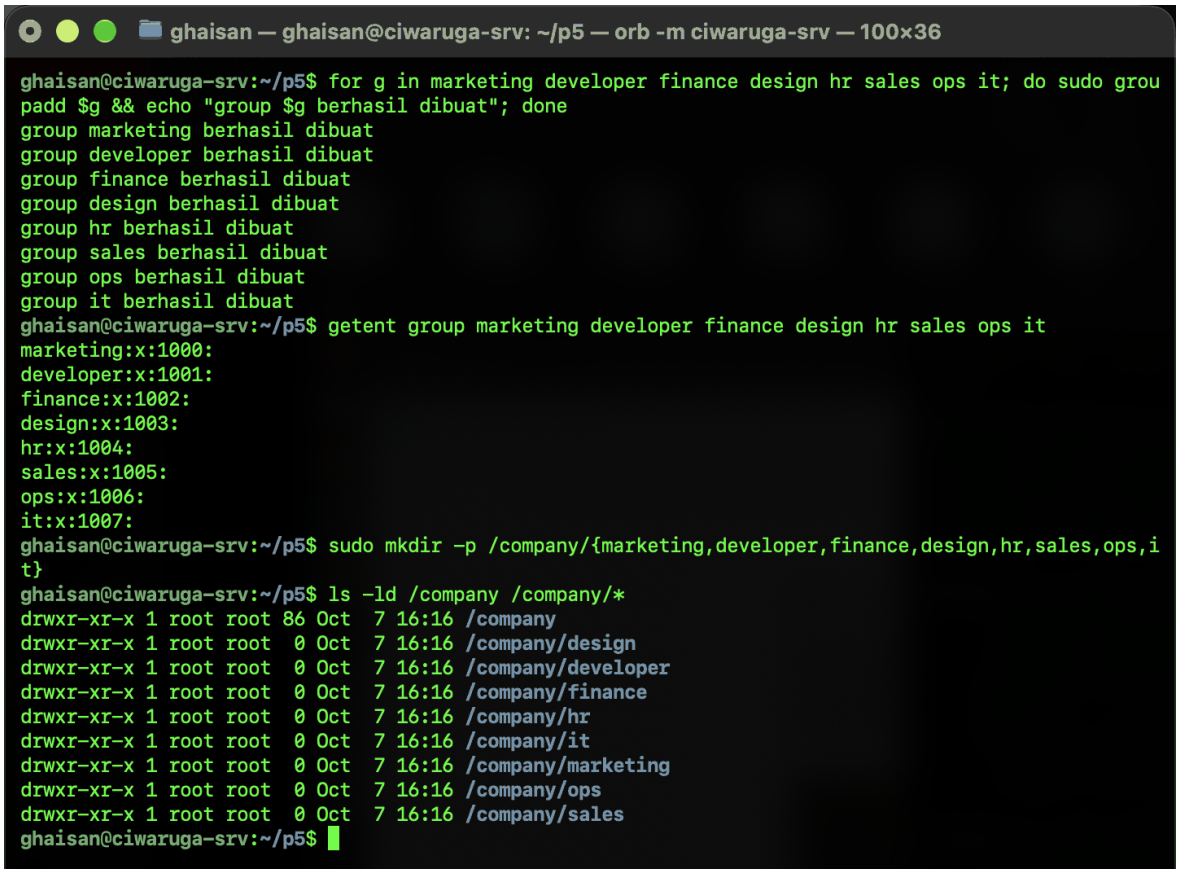
BAB II MEMBUAT GROUP DAN FOLDER GROUP

2.1 Instruksi 1: Delapan Group dan Folder Group

Command line.

```
for g in marketing developer finance design hr sales ops it; do sudo groupadd $g &&
echo "group $g berhasil dibuat"; done
getent group marketing developer finance design hr sales ops it
sudo mkdir -p /company/{marketing,developer,finance,design,hr,sales,ops,it}
ls -ld /company /company/*
```

Penjelasan. Perulangan `for` menjalankan `groupadd` untuk delapan nama group dan mencetak pesan setiap group berhasil dibuat. `getent group` memastikan kedelapan group sudah terdaftar di `/etc/group` (GID 1000 sampai 1007, belum ada anggota). Folder dibuat sekaligus dengan `mkdir -p` dan brace expansion `{marketing,developer,...}`, yang oleh bash dipecah menjadi delapan path. Opsi `-p` membuat `/company` sekalian. Hasil `ls -ld` menunjukkan kedelapan folder masih milik `root:root` dengan mode bawaan 755, jadi masih bisa dibuka siapa saja. Permission-nya diatur di instruksi 2.



Gambar 2.1: membuat delapan group dan folder group di /company

BAB III MENGATUR PERMISSION

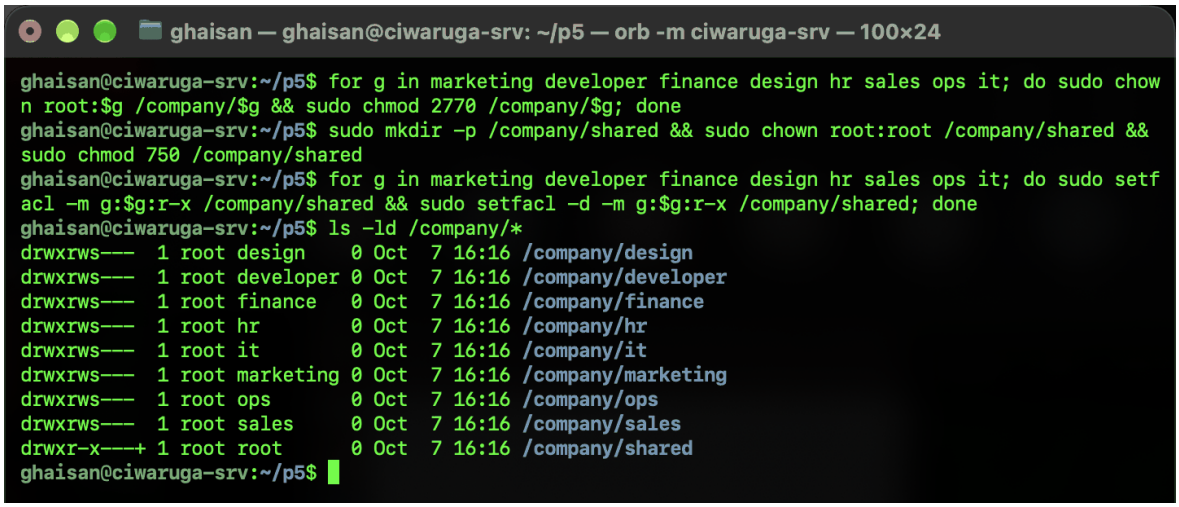
3.1 Instruksi 2: Folder Group dan Folder Shared

Command line.

```
for g in marketing developer finance design hr sales ops it; do sudo chown root:$g /
company/$g && sudo chmod 2770 /company/$g; done
sudo mkdir -p /company/shared && sudo chown root:root /company/shared && sudo chmod
750 /company/shared
for g in marketing developer finance design hr sales ops it; do sudo setfacl -m g:$g
:r-x /company/shared && sudo setfacl -d -m g:$g:r-x /company/shared; done
ls -ld /company/*
```

Penjelasan. Perulangan pertama mengubah grup tiap folder menjadi group yang sesuai, lalu memberi mode `2770`. Angka `770` berarti owner (root) dan group punya `rwx`, sedangkan other `---`, jadi hanya anggota group itu yang bisa masuk ke foldernya. Angka `2` di depan adalah bit setgid (terlihat sebagai `s` pada `drwxrws---`), supaya file yang dibuat anggota mana pun otomatis bergrup sama dengan foldernya dan bisa diakses rekan satu group.

Folder `shared` dibuat milik `root:root` dengan mode `750`, lalu setiap group diberi entri ACL `r-x` (`setfacl -m`). Hak `r` dan `x` cukup untuk melihat daftar isi dan membuka file, tetapi tanpa `w` tidak ada group yang bisa menambah, mengubah, atau menghapus isi shared, jadi aksesnya read-only. Opsi `-d` membuat default ACL, sehingga file dan folder yang nanti ditaruh admin di shared juga otomatis bisa dibaca semua group. Tanda `+` pada `drwxr-x---+` menunjukkan folder shared punya ACL.

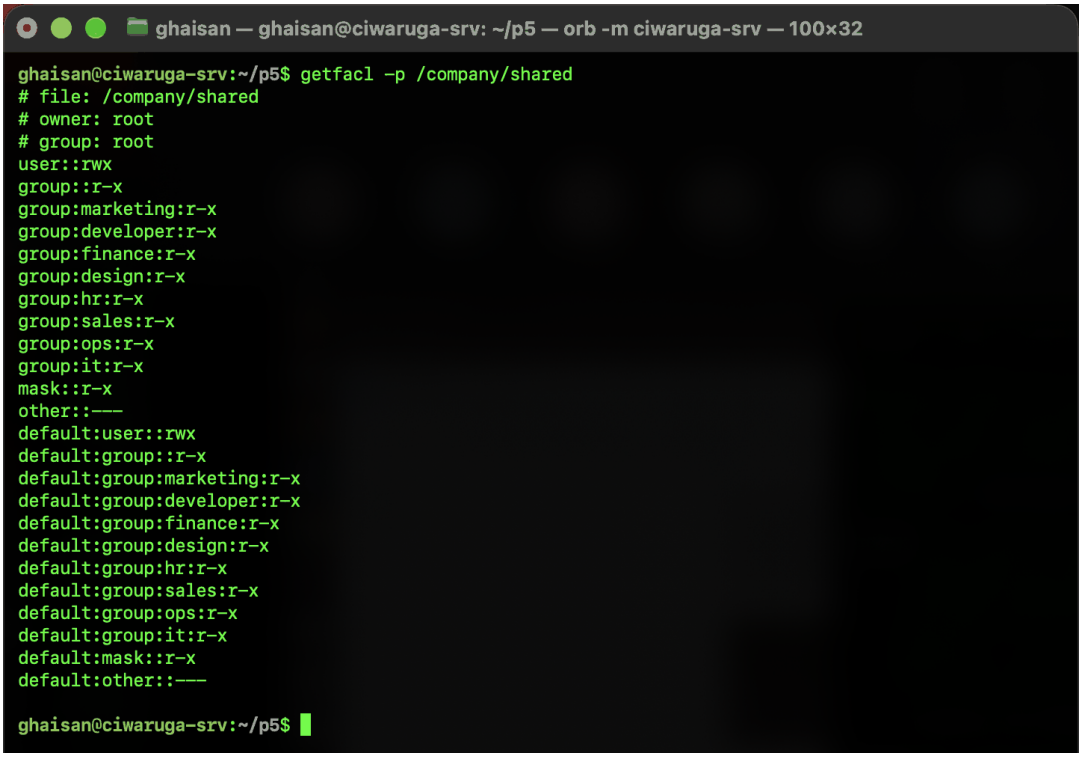


Gambar 3.1: mengatur pemilik, mode 2770, dan ACL read-only folder shared

Command line (verifikasi).

```
getfacl -p /company/shared
```

Penjelasan. `getfacl -p` menampilkan seluruh ACL folder shared: delapan entri `group:<nama>:r-x`, `mask::r-x` yang membatasi hak maksimal semua entri group menjadi baca saja, `other::---`, dan versi `default:` dari semua entri tersebut.



Gambar 3.2: ACL folder /company/shared

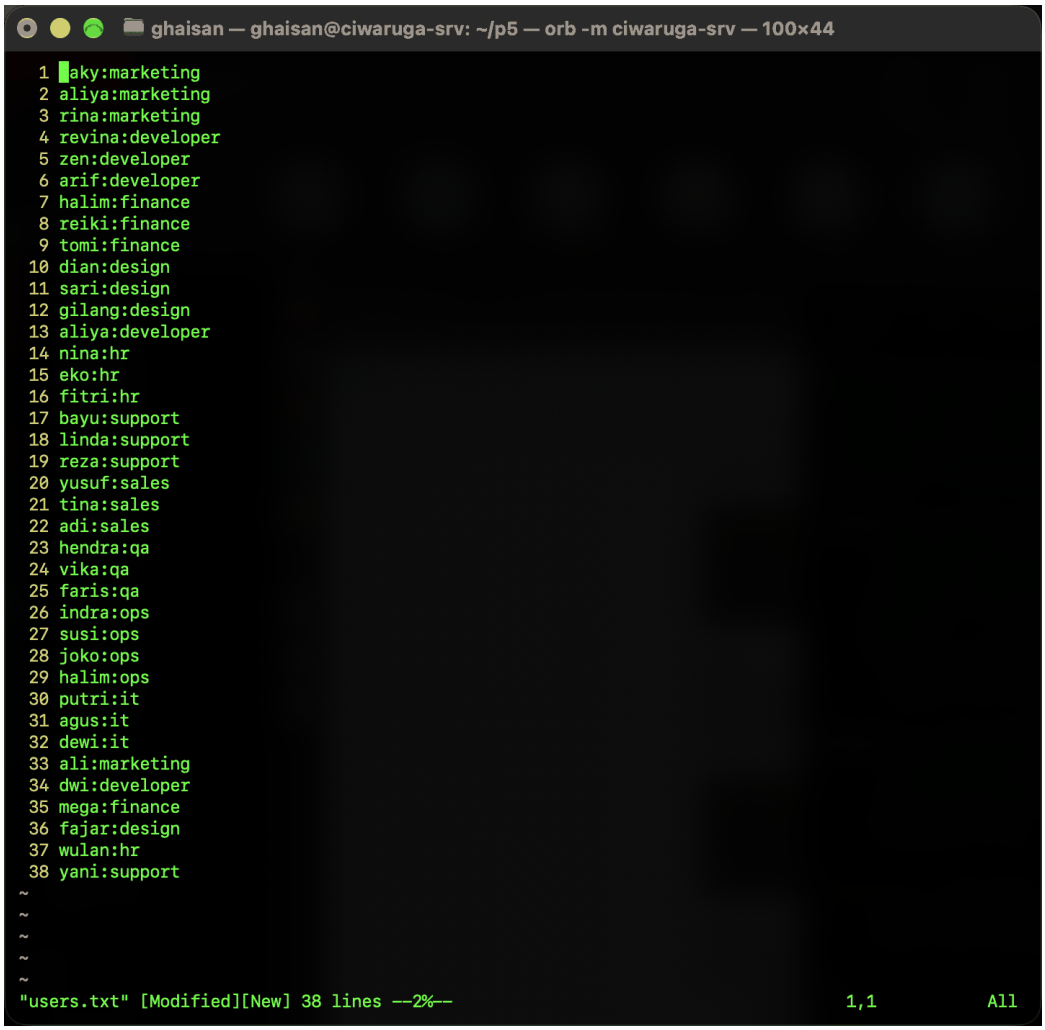
BAB IV SHELL SCRIPT CREATE USER

4.1 Membuat users.txt dengan vi

Command line.

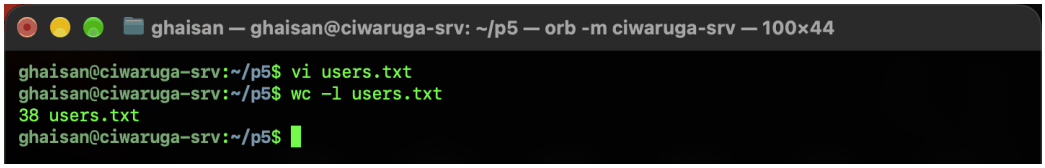
```
vi users.txt
wc -l users.txt
```

Penjelasan. File `users.txt` dibuat lebih dulu dengan vi. Di dalam vi, `:set paste` mematikan indentasi otomatis, `i` masuk ke mode insert, lalu 38 baris berformat `user:group` diketik sesuai soal. Setelah menekan Esc, `:set number` menampilkan nomor baris untuk pengecekan, dan `:wq` menyimpan lalu keluar. Isi file sengaja memuat tiga kasus yang harus ditangani script: user yang namanya dobel (aliya di baris 2 dan 13, halim di baris 7 dan 29) dan group yang belum ada (support di baris 17 dan qa di baris 23).



Gambar 4.1: isi users.txt di vi, 38 baris

Setelah disimpan, `wc -l` memastikan file berisi 38 baris.



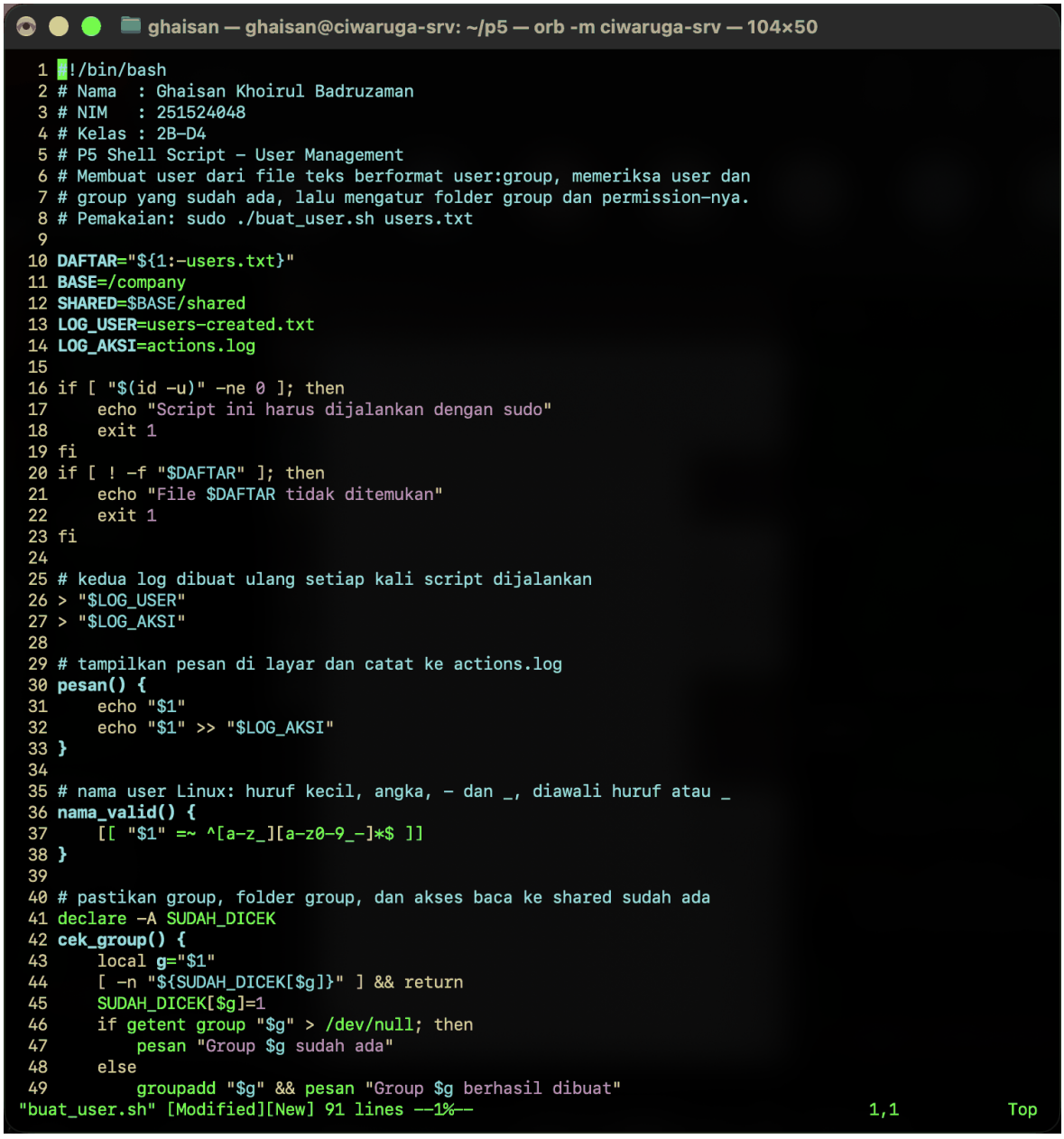
Gambar 4.2: users.txt tersimpan dengan 38 baris

4.2 Menulis Script dengan vi

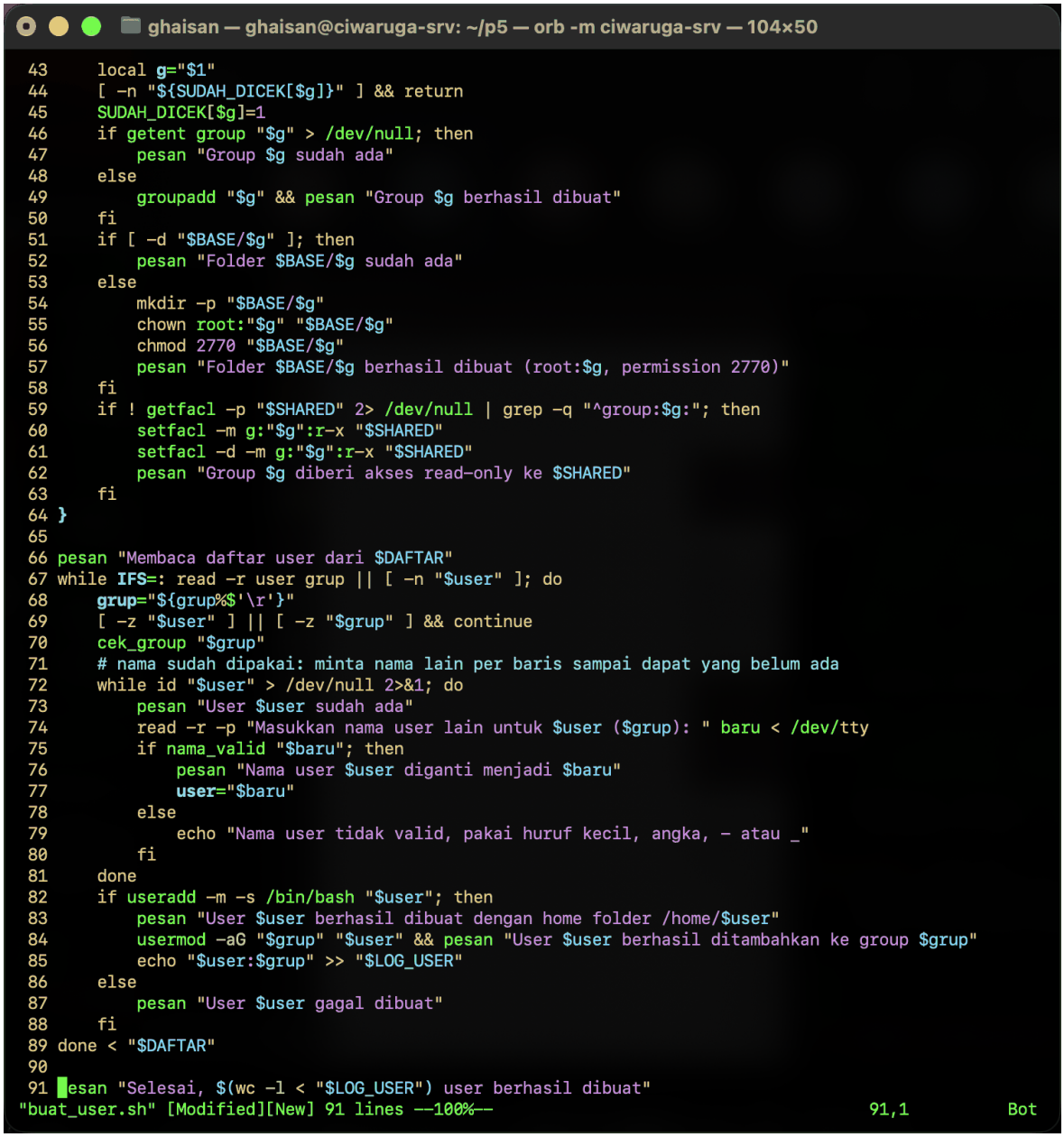
Command line.

```
vi buat_user.sh
chmod +x buat_user.sh
ls -l buat_user.sh users.txt
```

Penjelasan. Script ditulis di vi dengan cara yang sama. Gambar 4.3 dan 4.4 menunjukkan isi script di vi dengan nomor baris, sebelum disimpan dengan `:wq`. `chmod +x` memberi hak eksekusi supaya script bisa dijalankan langsung sebagai `./buat_user.sh`.

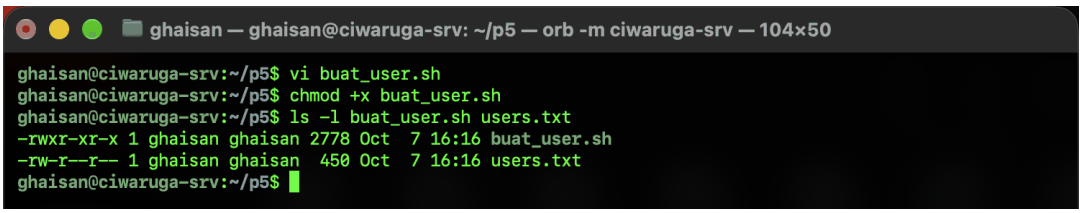


Gambar 4.3: buat_user.sh di vi, baris 1 sampai 49



Gambar 4.4: buat_user.sh di vi, baris 43 sampai 91

Hasil `ls -l` menunjukkan `buat_user.sh` sudah punya hak eksekusi (`-rwxr-xr-x`).



Gambar 4.5: script disimpan dan diberi hak eksekusi

4.3 Isi Script dan Cara Kerjanya

Script lengkapnya adalah sebagai berikut.

```
#!/bin/bash
# Nama  : Ghaisan Khoirul Badruzaman
# NIM   : 251524048
# Kelas : 2B-D4
# P5 Shell Script - User Management
# Membuat user dari file teks berformat user:group, memeriksa user dan
# group yang sudah ada, lalu mengatur folder group dan permission-nya.
# Pemakaian: sudo ./buat_user.sh users.txt

DAFTAR="${1:-users.txt}"
BASE=/company
SHARED=$BASE/shared
LOG_USER=users-created.txt
LOG_AKSI=actions.log

if [ "$(id -u)" -ne 0 ]; then
    echo "Script ini harus dijalankan dengan sudo"
    exit 1
fi
if [ ! -f "$DAFTAR" ]; then
    echo "File $DAFTAR tidak ditemukan"
    exit 1
fi

# kedua log dibuat ulang setiap kali script dijalankan
> "$LOG_USER"
> "$LOG_AKSI"

# tampilkan pesan di layar dan catat ke actions.log
pesan() {
    echo "$1"
    echo "$1" >> "$LOG_AKSI"
}

# nama user Linux: huruf kecil, angka, - dan _, diawali huruf atau _
nama_valid() {
    [[ "$1" =~ ^[a-z_][a-z0-9_-]*$ ]]
}

# pastikan group, folder group, dan akses baca ke shared sudah ada
declare -A SUDAH_DICEK
cek_group() {
    local g="$1"
    [ -n "${SUDAH_DICEK[$g]}" ] && return
    SUDAH_DICEK[$g]=1
    if getent group "$g" > /dev/null; then
        pesan "Group $g sudah ada"
    else
        groupadd "$g" && pesan "Group $g berhasil dibuat"
    fi
    if [ -d "$BASE/$g" ]; then
        pesan "Folder $BASE/$g sudah ada"
    else
        mkdir -p "$BASE/$g"
        chown root:"$g" "$BASE/$g"
```

```
56          chmod 2770 "$BASE/$g"
57          pesan "Folder $BASE/$g berhasil dibuat (root:$g, permission 2770)"
58      fi
59      if ! getfacl -p "$SHARED" 2> /dev/null | grep -q "^group:$g:"; then
60          setfacl -m g:"$g":r-x "$SHARED"
61          setfacl -d -m g:"$g":r-x "$SHARED"
62          pesan "Group $g diberi akses read-only ke $SHARED"
63      fi
64  }
65
66  pesan "Membaca daftar user dari $DAFTAR"
67  while IFS=: read -r user grup || [ -n "$user" ]; do
68      grup="${grup%$'\r'}"
69      [ -z "$user" ] || [ -z "$grup" ] && continue
70      cek_group "$grup"
71      # nama sudah dipakai: minta nama lain per baris sampai dapat yang belum ada
72      while id "$user" > /dev/null 2>&1; do
73          pesan "User $user sudah ada"
74          read -r -p "Masukkan nama user lain untuk $user ($grup): " baru < /dev/t
ty
75          if nama_valid "$baru"; then
76              pesan "Nama user $user diganti menjadi $baru"
77              user="$baru"
78          else
79              echo "Nama user tidak valid, pakai huruf kecil, angka, - atau _"
80          fi
81      done
82      if useradd -m -s /bin/bash "$user"; then
83          pesan "User $user berhasil dibuat dengan home folder /home/$user"
84          usermod -aG "$grup" "$user" && pesan "User $user berhasil ditambahkan ke
 group $grup"
85          echo "$user:$grup" >> "$LOG_USER"
86      else
87          pesan "User $user gagal dibuat"
88      fi
89  done < "$DAFTAR"
90
91  pesan "Selesai, $(wc -l < "$LOG_USER") user berhasil dibuat"
```

Setiap ketentuan soal dijawab oleh bagian script berikut.

Tabel 4.1: Ketentuan soal dan bagian script yang menjawabnya

Ketentuan	Baris	Cara kerja
a) Home folder	82	`useradd -m` membuat `/home/<user>` beserta isi bawaan dari `/etc/skel`, dan `-s /bin/bash` memberi shell bash.
b) Acuan users.txt	67, 89	`while IFS=: read -r user grup` membaca file per baris dan memecahnya di tanda `:` menjadi nama user dan group. `|| [ -n "$user" ]` menjaga baris terakhir tetap terbaca walaupun file tidak diakhiri newline, dan `${grup%$'\r'}` membuang karakter CR kalau file pernah diedit di Windows.

Ketentuan	Baris	Cara kerja
c) Existing user	72-81	`id "$user"` berhasil kalau user sudah ada. Selama nama sudah dipakai, script menampilkan pesan dan meminta nama lain per baris itu. Input dibaca dari `/dev/tty` (keyboard), bukan dari stdin, karena stdin perulangan sedang dipakai membaca users.txt. Nama baru dicek dengan `nama_valid` (huruf kecil, angka, - dan _), dan pertanyaan diulang kalau nama tidak valid atau ternyata juga sudah dipakai.
d) Existing group	42-64	`cek_group` memeriksa group dengan `getent group` dan folder dengan `[ -d ]`. Yang belum ada dibuat dengan `groupadd`, `mkdir`, `chown root:<group>`, dan `chmod 2770`, lalu group baru ditambahkan ke ACL read-only folder shared. Array asosiatif `SUDAH_DICEK` membuat tiap group hanya diperiksa sekali.
e) Pesan setiap aksi	30-33	Fungsi `pesan` mencetak pesan ke layar dan sekaligus menambahkannya ke actions.log, jadi setiap aksi (group dibuat, folder dibuat, user dibuat, user masuk group, nama diganti) pasti tercatat.
f) Dua file log	26-27, 85	Kedua log dikosongkan di awal setiap kali script dijalankan. users-created.txt hanya ditulis setelah `useradd` berhasil, dengan format `<user>:<group>`. actions.log hanya diisi lewat fungsi `pesan`, sehingga teks pertanyaan nama baru dan peringatan input tidak valid tidak ikut masuk ke log.

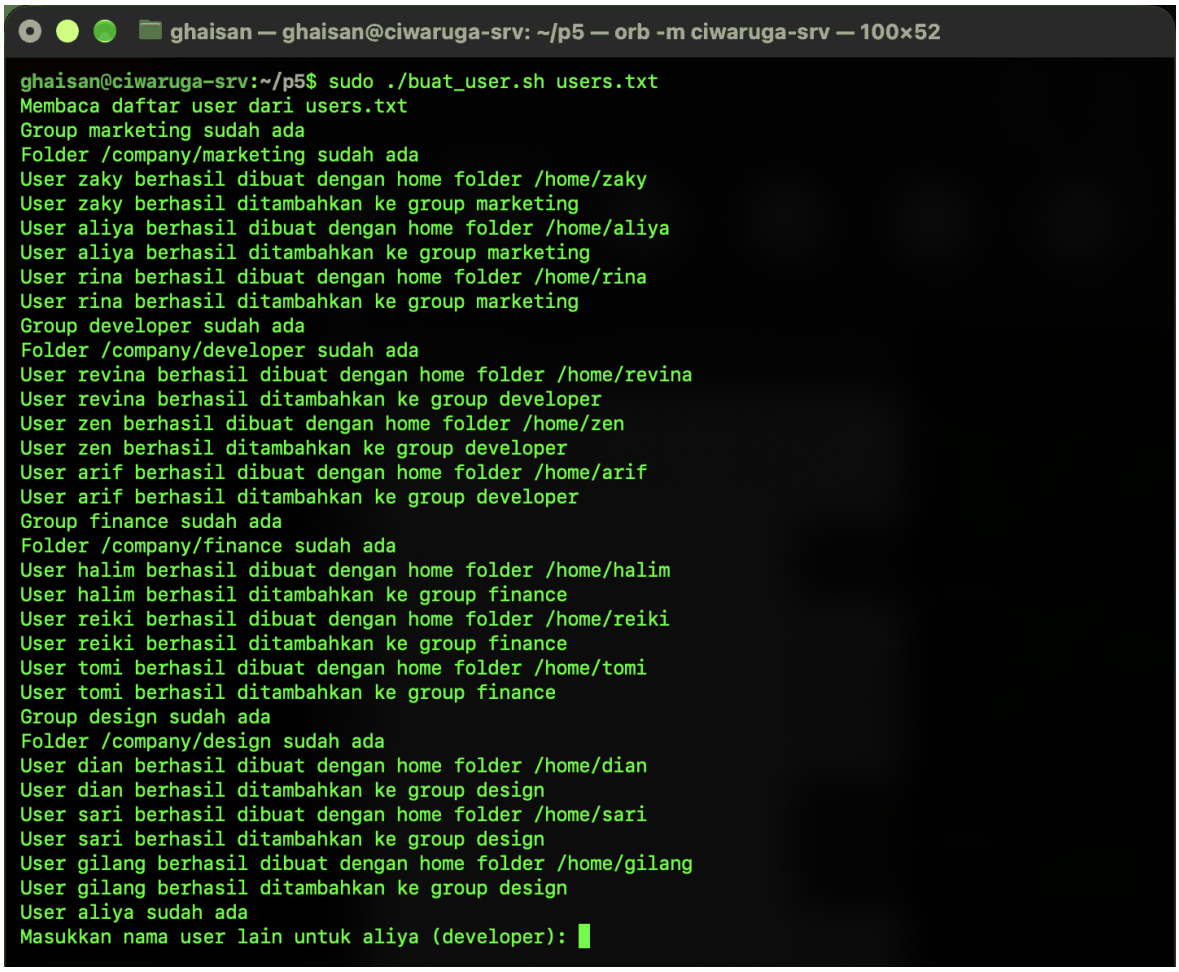
User baru dimasukkan ke group divisinya sebagai group tambahan (`usermod -aG`), sedangkan group utamanya tetap group pribadi bernama sama dengan user yang dibuat otomatis oleh `useradd` di Ubuntu. Pembuatan user dan penambahan ke group dipisah menjadi dua perintah supaya masing-masing punya pesan sendiri, sesuai contoh pesan di soal. Script juga menolak dijalankan tanpa sudo dan berhenti kalau file daftar user tidak ditemukan.

4.4 Menjalankan Script

Command line.

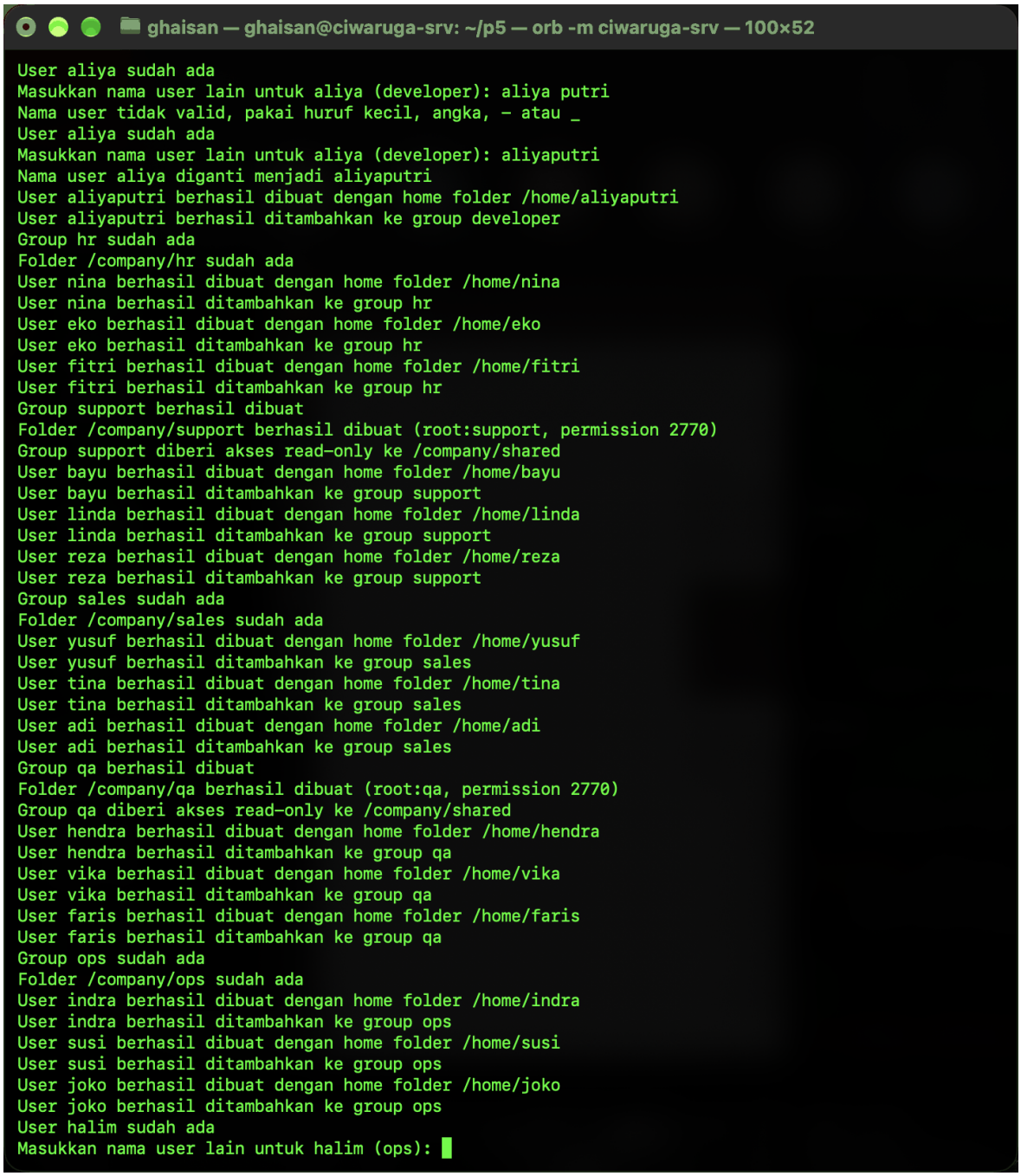
```
sudo ./buat_user.sh users.txt
```

Penjelasan. Script dijalankan dengan sudo karena `groupadd`, `useradd`, dan `setfacl` butuh hak root. Baris 1 sampai 12 (marketing, developer, finance, design) berjalan tanpa hambatan: group dan folder-nya sudah ada dari instruksi 1, jadi hanya dicatat "sudah ada", lalu tiap user dibuat dan dimasukkan ke group. Di baris 13 (`aliya:developer`) script menemukan aliya sudah dibuat di baris 2 dan berhenti meminta nama lain.



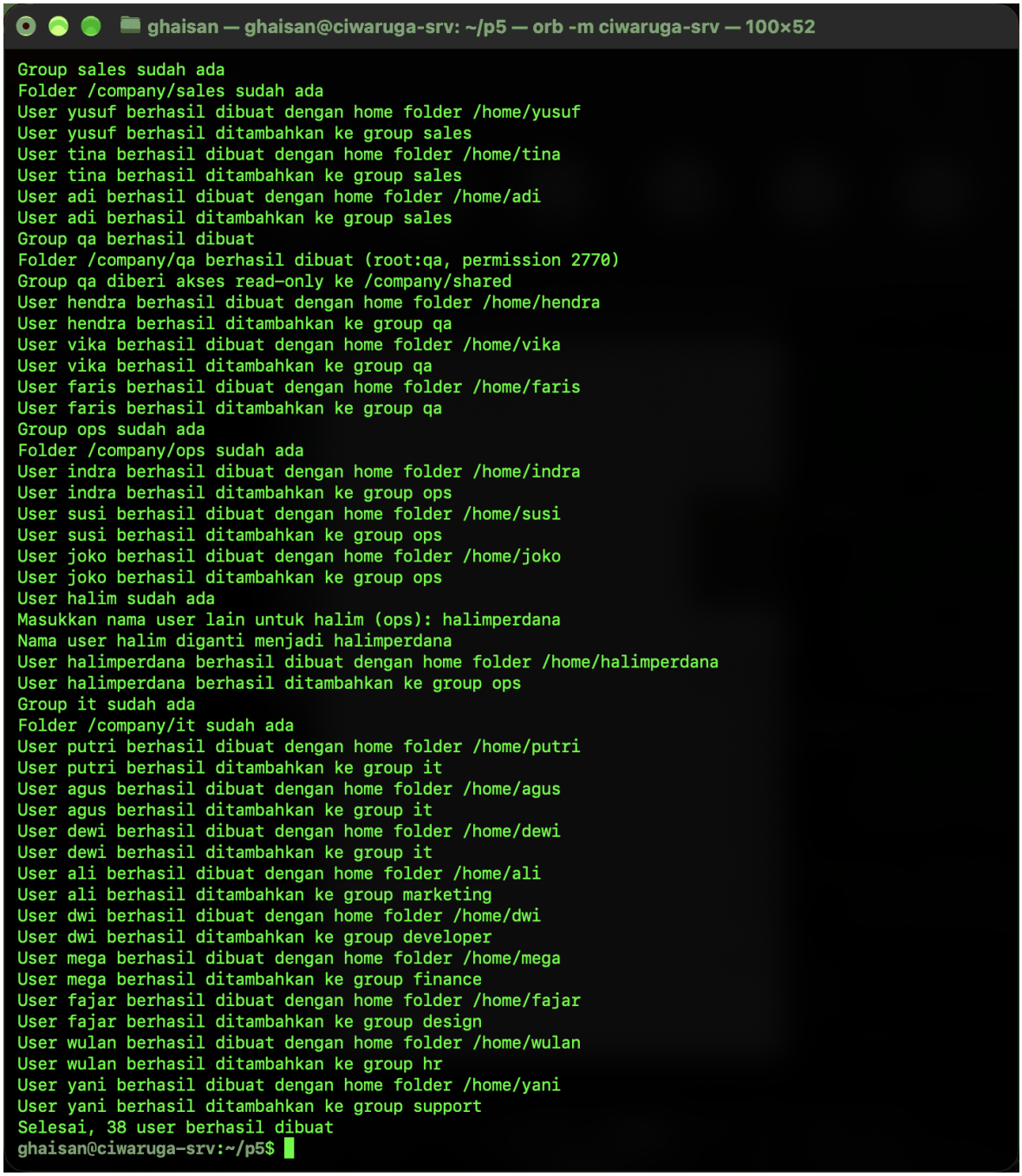
Gambar 4.6: script berhenti di baris aliya:developer dan meminta nama lain

Input pertama, `aliya putri`, ditolak karena mengandung spasi, lalu pertanyaan diulang. Input kedua, `aliyaputri`, diterima: user aliyaputri dibuat dan dimasukkan ke group developer. Selanjutnya terlihat pemeriksaan group (ketentuan d): group support dan qa belum ada, jadi script membuat group, foldernya dengan `root:<group>` dan mode 2770, lalu memberi kedua group akses read-only ke shared. Di baris 29 (`halim:ops`) script kembali meminta nama lain karena halim sudah dibuat di baris 7.



Gambar 4.7: nama aliyaputri dipakai, group support dan qa dibuat otomatis

Setelah `halimperdana` dimasukkan, sisa daftar diproses sampai selesai. Total 38 baris menghasilkan 38 user baru, dua di antaranya dengan nama pengganti. User ali, dwi, mega, fajar, wulan, dan yani di akhir daftar langsung dibuat tanpa pesan pemeriksaan group lagi, karena group mereka sudah diperiksa sebelumnya.



Gambar 4.8: script selesai, 38 user berhasil dibuat

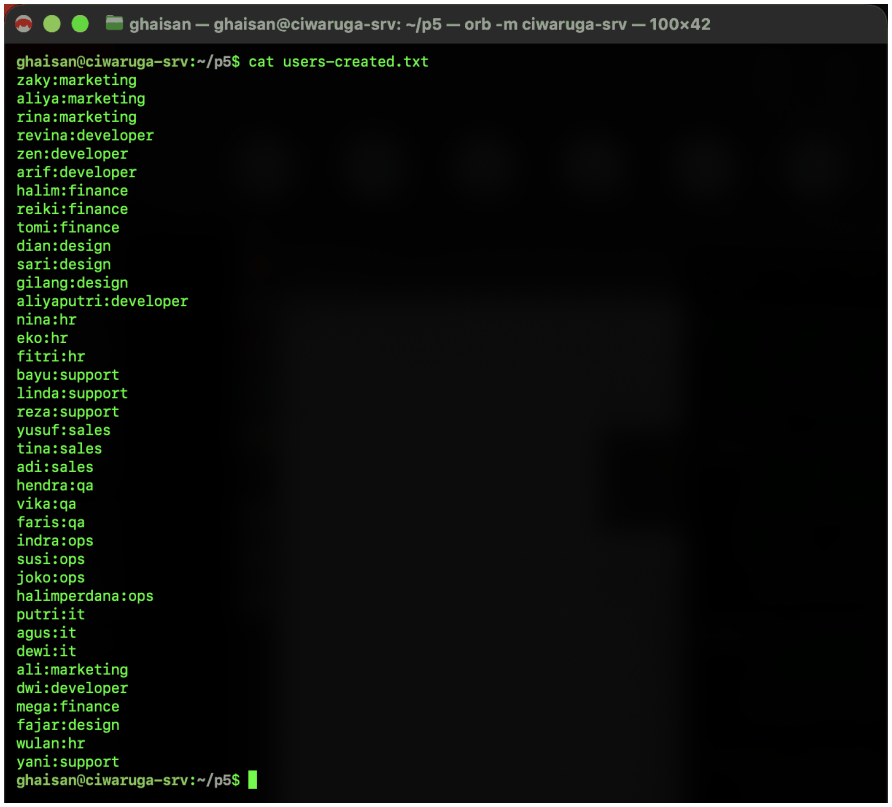
BAB V MEMERIKSA HASIL

5.1 File users-created.txt

Command line.

```
cat users-created.txt
```

Penjelasan. File berisi 38 baris berformat `<user>:<group>`, hanya untuk user yang benar-benar dibuat. Baris aliya dan halim yang kedua tercatat dengan nama penggantinya, yaitu `aliyaputri:developer` dan `halimperdana:ops`, bukan nama dari users.txt.



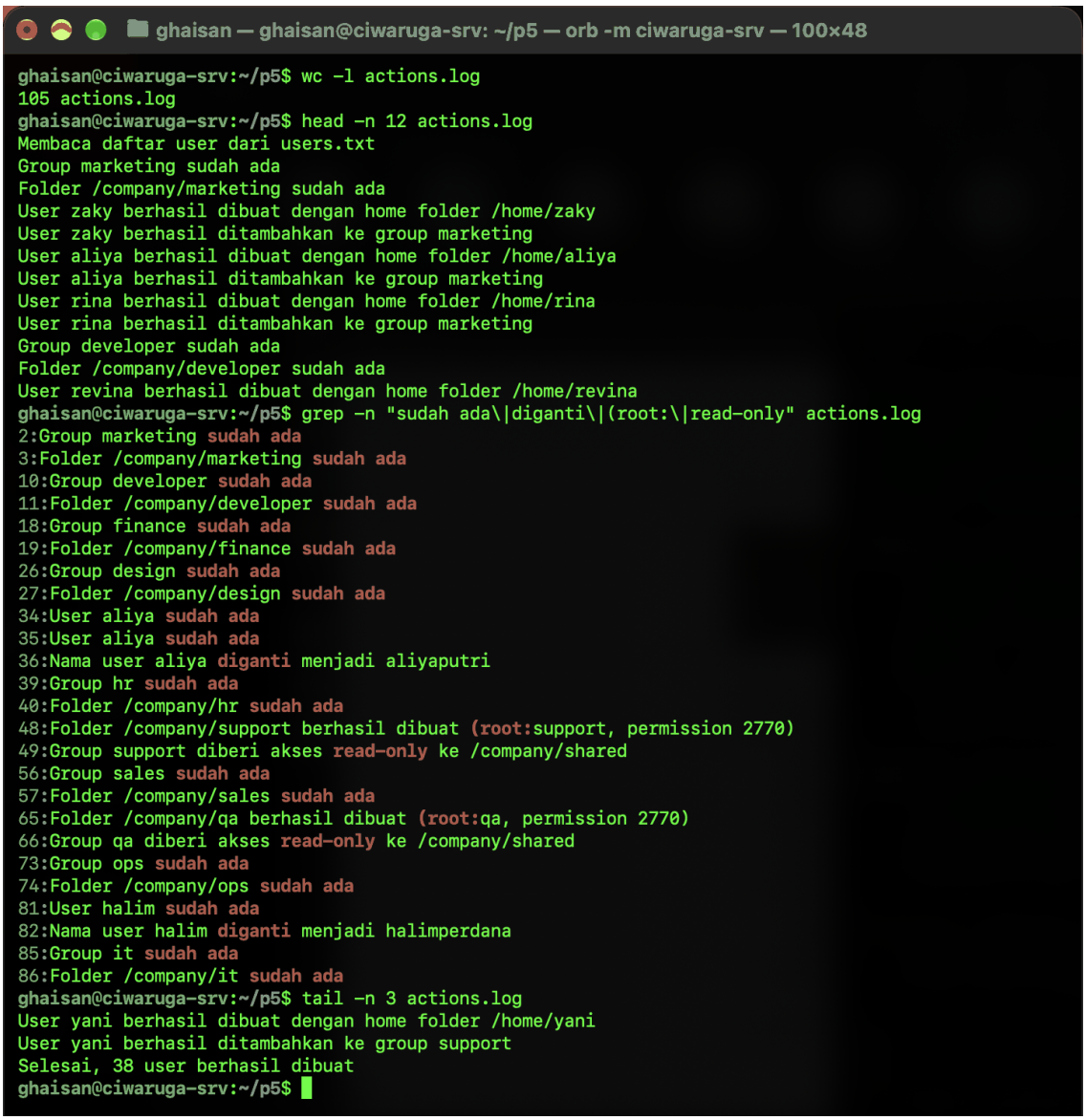
Gambar 5.1: isi users-created.txt

5.2 File actions.log

Command line.

```
wc -l actions.log
head -n 12 actions.log
grep -n "sudah ada\|diganti\|(root:\|read-only" actions.log
tail -n 3 actions.log
```

Penjelasan. actions.log berisi 105 pesan, persis sama dengan pesan yang tampil di layar saat script berjalan, tanpa teks pertanyaan dan tanpa peringatan input tidak valid. `head` menampilkan awal log, `grep -n` menyaring pesan pemeriksaan (group dan folder yang sudah ada, nama yang diganti, folder dan akses shared untuk group baru) beserta nomor barisnya, dan `tail` menampilkan ringkasan di akhir log.



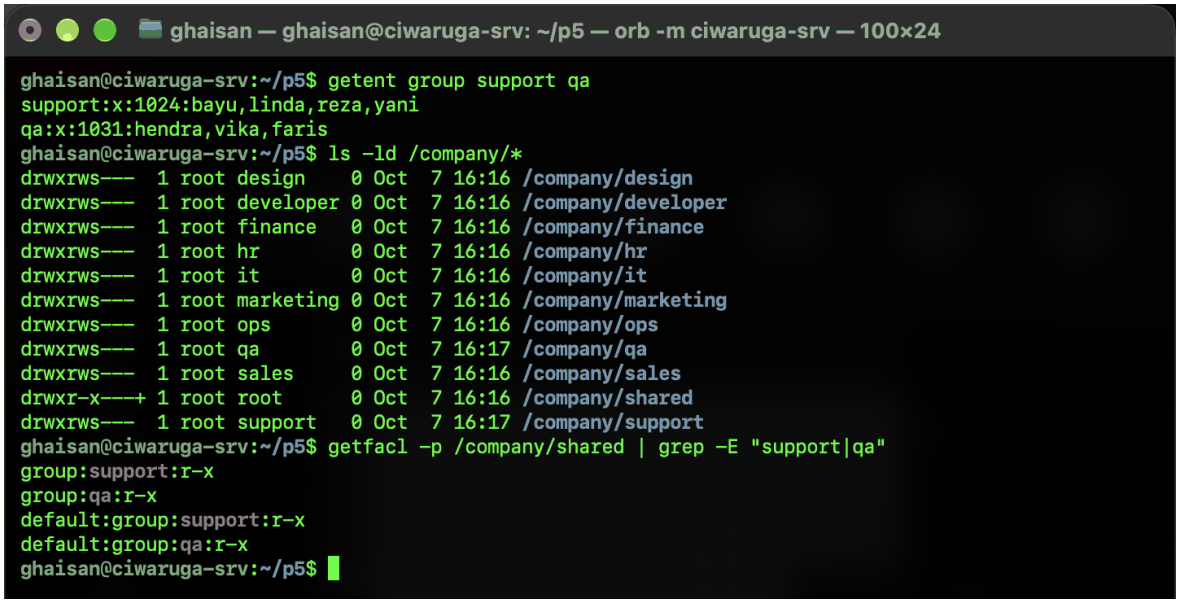
Gambar 5.2: isi actions.log

5.3 Group dan Folder Baru

Command line.

```
getent group support qa
ls -ld /company/*
getfacl -p /company/shared | grep -E "support|qa"
```

Penjelasan. Group support dan qa yang dibuat script sudah berisi anggotanya. Sekarang ada sepuluh folder group, semuanya dengan pola yang sama (`drwxrws---`, `root:<group>`), dan ACL folder shared sudah memuat entri read-only untuk support dan qa beserta default-nya.



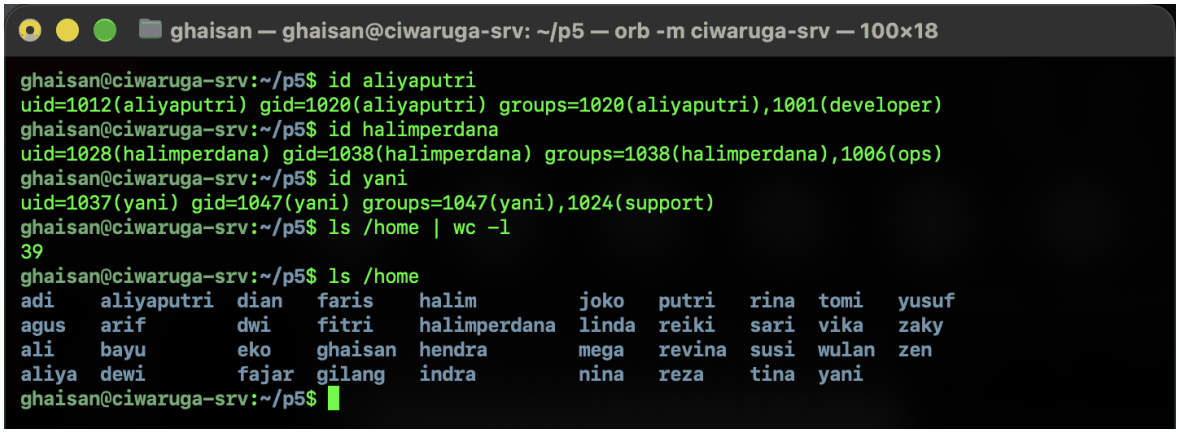
Gambar 5.3: group dan folder support serta qa hasil script

5.4 User dan Home Folder

Command line.

```
id aliyaputri
id halimperdana
id yani
ls /home | wc -l
ls /home
```

Penjelasan. `id` menunjukkan aliyaputri masuk group developer, halimperdana masuk ops, dan yani masuk support. Folder `/home` berisi 39 home folder, yaitu 38 user dari script ditambah akun admin ghaisan.



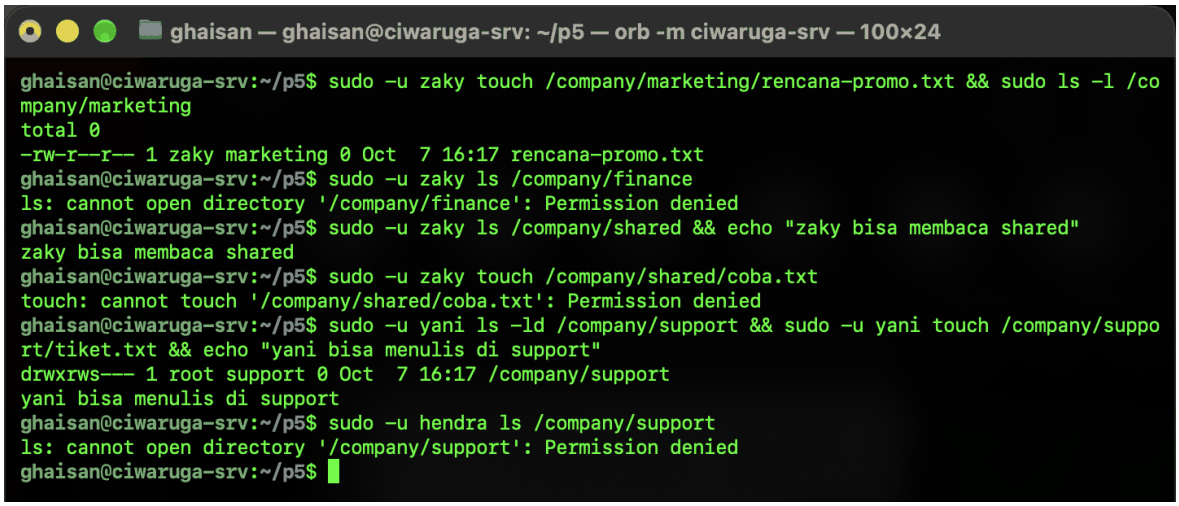
Gambar 5.4: grup user hasil script dan daftar home folder

5.5 Uji Hak Akses

Command line.

```
sudo -u zaky touch /company/marketing/rencana-promo.txt && sudo ls -l /company/marke
ting
sudo -u zaky ls /company/finance
sudo -u zaky ls /company/shared && echo "zaky bisa membaca shared"
sudo -u zaky touch /company/shared/coba.txt
sudo -u yani ls -ld /company/support && sudo -u yani touch /company/support/tiket.tx
t && echo "yani bisa menulis di support"
sudo -u hendra ls /company/support
```

Penjelasan. Pengujian dijalankan sebagai user lain dengan `sudo -u`. zaky (marketing) bisa membuat file di folder marketing, dan file itu otomatis bergrup marketing karena setgid. zaky ditolak membuka folder finance, bisa membaca folder shared, tetapi tidak bisa menulis di shared. yani (support, group yang dibuat script) bisa menulis di folder support, sedangkan hendra (qa) ditolak masuk ke folder support. Artinya aturan instruksi 2 juga berlaku untuk group yang dibuat script.



Gambar 5.5: uji akses folder group dan shared

BAB VI PENUTUP

6.1 Ringkasan Hasil

Tabel 6.1: Ringkasan hasil script

Hal	Hasil
Baris di users.txt	38
User berhasil dibuat	38 (aliyaputri dan halimperdana dengan nama pengganti)
Group	10: delapan dari instruksi 1, ditambah support dan qa dari script
Folder di /company	11: sepuluh folder group dan shared
Pesan di actions.log	105

6.2 Lesson Learned

1. Di dalam `while read ... done < users.txt`, stdin perulangan adalah file itu sendiri. Kalau pertanyaan nama baru dibaca dengan `read` biasa, yang terbaca justru baris berikutnya dari users.txt. Input dari keyboard harus diambil dari `/dev/tty`.
2. Input pengguna perlu divalidasi. Tanpa `nama_valid`, nama berspasi seperti `aliya putri` akan diteruskan ke `useradd` dan baru gagal di sana, sehingga baris itu terlewat tanpa user.
3. Satu fungsi `pesan` untuk layar dan log membuat kedua keluaran selalu sama. Pesan yang hanya bersifat interaksi (pertanyaan dan peringatan input) sengaja tidak lewat fungsi itu supaya log tetap berisi aksi saja.
4. Pemeriksaan group sebaiknya dilakukan sekali per group. Tanpa `SUDAH_DICEK`, setiap baris akan mencetak "Group sudah ada" dan log membengkak tanpa menambah informasi.
5. Script yang membuat group baru harus ikut membuat folder dan permission-nya dengan aturan yang sama dengan instruksi 2, termasuk ACL folder shared. Kalau tidak, group support dan qa akan punya aturan akses yang berbeda dari group lain.
6. Menulis script di vi lebih nyaman dengan `:set paste` (indentasi tidak bertumpuk saat mengetik blok) dan `:set number` (nomor baris memudahkan mencocokkan pesan error bash dengan baris script).

6.3 Kesimpulan

Shell script membuat pengelolaan user yang berulang menjadi konsisten dan bisa diaudit. Dari satu file daftar, 38 user dibuat lengkap dengan home folder dan group-nya, dua nama yang bentrok diselesaikan lewat input per baris, dua group baru dibuat otomatis dengan permission yang sama seperti group lain, dan setiap aksi tercatat di layar maupun di actions.log. Hasilnya diperiksa ulang dengan getent, getfacl, id, dan uji akses sebagai user lain, dan semuanya sesuai aturan soal.